One Pricer a Day

Neel Dev Sen

July 2026

Contents

1 Introduction **3**

2 European Options **4**

2.1 Day 1: Black Scholes 4

2.1.1 The Black Scholes PDE 4

2.1.2 Closed-Form Solution for European Call Options 5

2.1.3 Closed-Form Solution for European Put Options 7

2.2 Day 2: Black-76 Equation 8

2.2.1 Black-76 Formula for Call Options 9

2.2.2 Black-76 Formula for Put Options 10

1 Introduction

This is a project where I develop derivative pricers using **C++** and occasionally **Python**. My goal is to do at least one a day and you can access all of my source files and header files here: **GitHub Repo**

2 European Options

2.1 Day 1: Black Scholes

The first option that I am going to be pricing is the European option using the closed-form Black-Scholes equation.

In **GitHub** the full **C++** script is on this link: **source file**
The **header file** (first listing) is on this link: **header file**

2.1.1 The Black Scholes PDE

The Black-Scholes equation takes the assumption of the **risk-neutral pricing measure**

$$dS_t = rS_t dt + \sigma S_t dW_t \tag{2.1}$$

[1] The Black Scholes equation is a partial differential equation that is used to model how option prices change as the initial stock price, time to maturity, risk-free interest rate and implied volatility change. The partial differential equation is shown below: where S_t is the price of the asset at a given time t, dS_t is the infinitesimal change in the asset's price, r is the risk-free rate of interest, dt is the infinitesimal change in time, σ is the implied volatility and dW_t is the infinitesimal change in the **Wiener F function**. The **Wiener function** is defined with the property:

$$W_T - W_t \sim \mathcal{N}(0, \sqrt{T-t}) \tag{2.2}$$

where $\mathcal{N}(\mu, \sigma)$ is the **normal distribution** with mean μ and standard-deviation σ.[2]

Using **stochastic calculus** and **Ito's lemma**, the Black-Scholes becomes the partial differential equation:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{2.3}$$

In this equation V is the option's price, t is the time to maturity.
$\frac{\partial V}{\partial t}$ is the rate of change of the option's price to time, $\frac{\partial V}{\partial S}$ is the rate of change of the option's price to the underlying stock price and $\frac{\partial^2 V}{\partial S^2}$ is a concavity factor on how the option price changes with the stock price. The Black-Scholes can be solved for a call option and a put option. [3].

2.1.2 Closed-Form Solution for European Call Options

The closed-form solution for the Black-Scholes equation where the option is a call option is:

$$C(t, S) = S\Phi(d_1) - Ke^{-r(T-t)}\Phi(d_2) \tag{2.4}$$

where:

$$d_1 = \frac{\ln(\frac{S}{K}) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}}$$

and:

$$d_2 = d_1 - \sigma\sqrt{T-t}$$

In this equation $T - t$ represents the time to maturity and K represents the strike price. [3].

$\Phi(z)$ is the standard normal cumulative distribution function which can be represented with the error function $\text{erf}(z)$:

$$\Phi(z) = \frac{1}{2}(1 + \text{erf}(\frac{z}{\sqrt{2}})) \tag{2.5}$$

In C++ this can be implemented as:

```
1  #include <iostream>
2  #include <cmath>
3  #include <type_traits>
4
5  template <typename T>
6  auto normalCDF(T x) -> std::common_type_t <double , T>
7  {
8          using commonType = std::common_type_t<double, T>;
9          return static_cast<commonType>(0.5) * (static_cast<
               commonType>(1) + std::erf(x / std::sqrt(2.0)));
10 }
```

The function we are creating is called **normalCDF**. This listing uses the **cmath** header file in order to import **std::erf**. It also uses a function template with a type placeholder **T** so that we can use this function for any type given. In order to ensure that this function has at least a type **double** we use the header file **type traits** which allows us to specify that we want this function to be at least double in line 6 trailing return type. In line 8 we ensure that the **commonType** alias used amongst all the literals and variables in this function are also at least of type **double**. This function returns the result shown in equation 2.5 with a type of at least **double**.

Now to implement the function shown in equation 2.4 in C++ we use:

```
1  #include "functions.hpp" // header file for normalCDF
2
3  template <typename SType, typename KType, typename rType,
       typename sigmaType, typename tType>
4  auto blackScholesCall(SType S, KType K, rType r, sigmaType
       sigma, tType t) -> std::common_type_t<double, SType, KType,
        rType, sigmaType, tType>
5  {
6          using commonType = std::common_type_t<double, SType,
               KType, rType, sigmaType, tType>;
7          auto S_new {static_cast<commonType>(S)};
8          auto K_new {static_cast<commonType>(K)};
9          auto r_new {static_cast<commonType>(r)};
10         auto sigma_new {static_cast<commonType>(sigma)};
11         auto t_new {static_cast<commonType>(t)};
12         auto sqrt_t_new {std::sqrt(t_new)};
13
14         auto d1 {(std::log(S_new / K_new) + (r_new + (
               static_cast<commonType>(0.5) * sigma_new *
               sigma_new) * t_new) / (sigma_new * sqrt_t_new)};
15         auto d2 {d1 - sigma_new * sqrt_t_new};
16
17         auto C {S_new * normalCDF(d1) - K_new * std::exp(-
               r_new * t_new) * normalCDF(d2)};
18         return C;
19 }
```

The function in this listing is called **blackScholesCall**. This listing again uses the same header files and the same principles as the one before and converts every thing into a **commonType** of at least type **double**. The formula for d_1 is shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in equation 2.4 is shown in line 16 using the previously defined function **normalCDF**. The function returns the value of the **call option** with at least type **double**.

2.1.3 Closed-Form Solution for European Put Options

The closed-form solution for the Black-Scholes equation fora put option is:

$$P(t, S) = Ke^{-r(T-t)}\Phi(-d_2) - S\Phi(-d_1) \tag{2.6}$$

using the same definitions of d_1 and d_2 in 2.4 [3]

The implementation of this in C++ is:

```
1  #include "functions.hpp" // header file for normalCDF
2
3  template <typename SType, typename KType, typename rType,
       typename sigmaType, typename tType>
4  auto blackScholesPut(SType S, KType K, rType r, sigmaType
       sigma, tType t) -> std::common_type_t<double, SType, KType,
        rType, sigmaType, tType>
5  {
6          using commonType = std::common_type_t<double, SType,
               KType, rType, sigmaType, tType>;
7          auto S_new {static_cast<commonType>(S)};
8          auto K_new {static_cast<commonType>(K)};
9          auto r_new {static_cast<commonType>(r)};
10         auto sigma_new {static_cast<commonType>(sigma)};
11         auto t_new {static_cast<commonType>(t)};
12         auto sqrt_t_new {std::sqrt(t_new)};
13
14         auto d1 {(std::log(S_new / K_new) + (r_new + (
               static_cast<commonType>(0.5) * sigma_new *
               sigma_new) * sqrt_t_new) / (sigma_new * sqrt_t_new)
               };
15         auto d2 {d1 - sigma_new * std::sqrt(t_new)};
16
17         auto P {K_new * std::exp(-r_new * t_new) * normalCDF(-
               d2) - S_new * normalCDF(-d1)};
18         return P;
19 }
```

The function in this listing is called **blackScholesPut**. This listing is almost identical to the one for **blackScholesCall**. The formula for d_1 is again shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in equation 2.6 is computed in line 16 (stored as **P**) using the function **normalCDF**. The function returns the value of the **put option** with at least type **double**.

2.2 Day 2: Black-76 Equation

The Black-76 model is just an adaption of the **Black-Scholes equation** except it uses the price of futures and bonds to price options.

In **GitHub** the source file can be found here: **Source File**

The modelling assumptions are that the futures of forward price process is modelled by **geometric Brownian motion** shown by the equation:

$$dF_t = \sigma F_t dW_t \tag{2.7}$$

where F_t is the price of a future or a forward at a given time, dF_t is the infinitesimal change of the price of a future of a forward, σ is implied volatility and dW_t is the infinitesimal change in the **Wiener function**.[4]

For the listings below I will continue using the header file **functions.hpp** that I defined on day 1. I will continue to use the **normalCDF** function from there.

2.2.1 Black-76 Formula for Call Options

The Black-76 Formula for Call options is:

$$C(F,t) = e^{-r(T-t)}(F\Phi(d_1) - K\Phi(d_2)) \tag{2.8}$$

where:

$$d_1 = \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}}$$

and:

$$d_2 = d_1 - \sigma\sqrt{T-t}$$

[4] The implementation in C++ is:

```
1  #include "functions.hpp" // header file for normalCDF
2
3  template <typename FType, typename KType, typename rType,
       typename sigmaType, typename tType>
4  auto black76Call(FType F, KType K, rType r, sigmaType sigma,
       tType t) -> std::common_type_t<double, FType, KType, rType,
        sigmaType, tType>
5  {
6          using commonType = std::common_type_t<double, FType,
               KType, rType, sigmaType, tType>;
7          auto F_new {static_cast<commonType>(F)};
8          auto K_new {static_cast<commonType>(K)};
9          auto r_new {static_cast<commonType>(r)};
10         auto sigma_new {static_cast<commonType>(sigma)};
11         auto t_new {static_cast<commonType>(t)};
12         auto sqrt_t_new {std::sqrt(t_new)};
13
14         auto d1 {(std::log(F_new / K_new) + (static_cast<
               commonType>(0.5) * sigma_new * sigma_new) * t_new )
                / (sigma_new * sqrt_t_new)};
15         auto d2 {d1 - sigma_new * sqrt_t_new};
16
17         auto C {(std::exp(-r_new * t_new)) * (F_new *
               normalCDF(d1) - K_new * normalCDF(d2))};
18         return C;
19 }
```

d_1 is calculated in line 13, d_2 is calculated in line 14 and finally equation 2.8 is computed in line 16 (stored as **C**) and returned in line 17.

2.2.2 Black-76 Formula for Put Options

The Black-76 Formula for Call options is:

$$P(F,t) = e^{-r(T-t)}(K\Phi(-d_2) - F\Phi(-d_1)) \tag{2.9}$$

where:

$$d_1 = \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}}$$

and:

$$d_2 = d_1 - \sigma\sqrt{T-t}$$

[4] The implementation in C++ is:

```
1  #include "functions.hpp" // header file for normalCDF
2
3  template <typename FType, typename KType, typename rType,
       typename sigmaType, typename tType>
4  auto black76Put(FType F, KType K, rType r, sigmaType sigma,
       tType t) -> std::common_type_t<double, FType, KType, rType,
        sigmaType, tType>
5  {
6          using commonType = std::common_type_t<double, FType,
               KType, rType, sigmaType, tType>;
7          auto F_new {static_cast<commonType>(F)};
8          auto K_new {static_cast<commonType>(K)};
9          auto r_new {static_cast<commonType>(r)};
10         auto sigma_new {static_cast<commonType>(sigma)};
11         auto t_new {static_cast<commonType>(t)};
12         auto sqrt_t_new {std::sqrt(t_new)};
13
14         auto d1 {(std::log(F_new / K_new) + (static_cast<
               commonType>(0.5) * sigma_new * sigma_new) * t_new )
                / (sigma_new * sqrt_t_new)};
15         auto d2 {d1 - sigma_new * sqrt_t_new};
16
17         auto P {std::exp(-r_new * t_new) * (K_new * normalCDF
               (-d2) - (F_new * normalCDF(-d1)))};
18         return P;
19 }
```

d_1 is calculated in line 13, d_2 is calculated in line 14 and finally equation 2.9 is computed in line 16 (stored as **P**) and returned in line 17.

References

[1] Shreve, S. (2004). *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer Science & Business Media.

[2] Lalley, S. P. (n.d.). *Brownian motion*. https://galton.uchicago.edu/~lalley/Courses/313/BrownianMotionCurrent.pdf

[3] Haugh, M. (2016). *The Black-Scholes model* (IEOR E4706: Foundations of Financial Engineering lecture notes). Columbia University. https://www.columbia.edu/~mh2078/FoundationsFE/BlackScholes.pdf

[4] Black, F. (1976). The pricing of commodity contracts. *Journal of Financial Economics*, *3*(1–2), 167–179. https://doi.org/10.1016/0304-405X(76)90024-6